| | |
|---|---|
| | **Curso Superior em Análise e Desenvolvimento de Sistemas** |
| | Disciplina: Projeto de Software |
| | Profª. Denise do Rocio Maciel |
| | **LISTA DE EXERCÍCIOS: DIAGRAMA DE CLASSES UML** |

Instruções: para cada código, desenhe o diagrama de classes no quadro, usando a notação UML vista em aula (+ public, - private, # protected, ~ pacote; losango vazado = agregação; losango preenchido = composição; seta tracejada «usa» = dependência; triângulo vazado = herança/realização).

Faculdade Senac – 2026 – Projeto de Software – Contexto: Estoque

1. Represente a classe Java abaixo em UML, com o nome da classe, os atributos (com tipos) e os métodos (com parâmetros e retorno), indicando a visibilidade de cada membro.

```
public class Produto {
    private String nome;
    private double preco;
    private int quantidade;

    public Produto(String nome, double preco) {
        this.nome = nome;
        this.preco = preco;
    }

    public String getNome() { return nome; }

    public double getPreco() { return preco; }

    public void setPreco(double preco) {
        this.preco = preco;
    }

    public double calcularTotal() {
        return preco * quantidade;
    }
}
```

Faculdade Senac – 2026 – Projeto de Software – Contexto: Casa inteligente

2. Represente a classe Java abaixo em UML. Atenção às quatro visibilidades usadas: public, private, protected e a de pacote (sem modificador).

```
public class Lampada {
    private boolean ligada;
    protected int potencia;
    String comodo;

    public Lampada(int potencia, String comodo) {
        this.potencia = potencia;
        this.comodo = comodo;
    }

    public void ligar() { ligada = true; }

    public void desligar() { ligada = false; }

    public boolean isLigada() { return ligada; }

    protected double consumo(int horas) {
        return potencia * horas / 1000.0;
    }

    void registrarUso() {
        System.out.println(comodo + " em uso");
    }
}
```

| | |
|---|---|
| | **Curso Superior em Análise e Desenvolvimento de Sistemas** |
| | Disciplina: Projeto de Software |
| | Profª. Denise do Rocio Maciel |
| | **LISTA DE EXERCÍCIOS: DIAGRAMA DE CLASSES UML** |

Faculdade Senac – 2026 – Projeto de Software – Contexto: Clima

3. Represente a classe Java abaixo em UML. Como se representam os membros static e a constante (final)?

```
public class Termometro {
    public static final double ZERO_ABSOLUTO =
        -273.15;
    private static int totalLeituras = 0;
    private double celsius;

    public Termometro(double celsius) {
        this.celsius = celsius;
        totalLeituras++;
    }

    public double getFahrenheit() {
        return celsius * 9 / 5 + 32;
    }

    public static int getTotalLeituras() {
        return totalLeituras;
    }
}
```

Faculdade Senac – 2026 – Projeto de Software – Contexto: Recursos Humanos

4. Represente o código Java abaixo em um diagrama de classes UML, com atributos, métodos, visibilidades, o tipo de relacionamento e as multiplicidades. Justifique a multiplicidade de cada ponta e explique como a visibilidade de setDono aparece no diagrama.

```
public class Funcionario {
    private String nome;
    private Cracha cracha;

    public Funcionario(String nome, Cracha c) {
        this.nome = nome;
        receberCracha(c);
    }

    public void receberCracha(Cracha c) {
        if (c.getDono() == this) return;
        if (c.getDono() != null)
            throw new IllegalStateException(
                "Crachá em uso");
        if (this.cracha != null)
            this.cracha.setDono(null);
        this.cracha = c;
        c.setDono(this);
    }

    public Cracha getCracha() { return cracha; }
}

public class Cracha {
    private int numero;
    private Funcionario dono;

    public Cracha(int numero) {
        this.numero = numero;
    }

    void setDono(Funcionario f) { dono = f; }
    public Funcionario getDono() { return dono; }
}
```

| | |
|---|---|
| | **Curso Superior em Análise e Desenvolvimento de Sistemas** |
| | Disciplina: Projeto de Software |
| | Profª. Denise do Rocio Maciel |
| | **LISTA DE EXERCÍCIOS: DIAGRAMA DE CLASSES UML** |

Faculdade Senac – 2026 – Projeto de Software – Contexto: Gestão acadêmica

5. Represente o código Java abaixo em um diagrama de classes UML, com atributos, métodos, visibilidades, o tipo de relacionamento e as multiplicidades. Qual trecho do código garante o mínimo 1 no lado da Turma?

```
import java.util.*;

public class Professor {
    private String nome;
    private List<Turma> turmas =
        new ArrayList<>();

    public Professor(String nome, Turma t) {
        this.nome = nome;
        assumirTurma(t);
    }

    public void assumirTurma(Turma t) {
        if (t.getProfessor() != null)
            throw new IllegalStateException(
                "Turma já tem professor");
        turmas.add(t);
        t.setProfessor(this);
    }

    public List<Turma> getTurmas() {
        return Collections
            .unmodifiableList(turmas);
    }
}

public class Turma {
    private String codigo;
    private Professor professor;

    public Turma(String codigo) {
        this.codigo = codigo;
    }

    void setProfessor(Professor p) {
        this.professor = p;
    }

    public Professor getProfessor() {
        return professor;
    }
}
```

Faculdade Senac – 2026 – Projeto de Software – Contexto: Matrícula

6. Represente o código Java abaixo em um diagrama de classes UML, com atributos, métodos, visibilidades, o tipo de relacionamento e as multiplicidades. Por que o teste contains() é indispensável nos dois métodos?

```
import java.util.*;

public class Aluno {
    private String matricula;
    private List<Disciplina> disciplinas =
        new ArrayList<>();

    public Aluno(String matricula) {
        this.matricula = matricula;
    }

    public void matricular(Disciplina d) {
        if (!disciplinas.contains(d)) {
            disciplinas.add(d);
            d.adicionarAluno(this);
        }
    }
}

public class Disciplina {
    private String nome;
    private List<Aluno> alunos =
        new ArrayList<>();

    public Disciplina(String nome) {
        this.nome = nome;
    }

    public void adicionarAluno(Aluno a) {
        if (!alunos.contains(a)) {
            alunos.add(a);
            a.matricular(this);
        }
    }
}
```

| | |
|---|---|
| | **Curso Superior em Análise e Desenvolvimento de Sistemas** |
| | Disciplina: Projeto de Software |
| | Profª. Denise do Rocio Maciel |
| | **LISTA DE EXERCÍCIOS: DIAGRAMA DE CLASSES UML** |

Faculdade Senac – 2026 – Projeto de Software – Contexto: Transporte

7. Represente o código Java abaixo em um diagrama de classes UML, com atributos, métodos, visibilidades, o tipo de relacionamento e as multiplicidades. Que linha do código revela o tipo de relacionamento? O que acontece com o Veiculo se m1 for descartado?

```
public class Veiculo {
    private String placa;
    private String modelo;

    public Veiculo(String placa, String modelo) {
        this.placa = placa;
        this.modelo = modelo;
    }

    public String getPlaca() { return placa; }
}

public class Motorista {
    private String nome;
    private Veiculo veiculo;

    public Motorista(String nome, Veiculo v) {
        this.nome = nome;
        this.veiculo = v;
    }

    public void dirigir() {
        System.out.println(nome + " dirige "
            + veiculo.getPlaca());
    }
}

// Uso
Veiculo v = new Veiculo("ABC1D23", "Gol");
Motorista m1 = new Motorista("Ana", v);
Motorista m2 = new Motorista("Rui", v);
```

Faculdade Senac – 2026 – Projeto de Software – Contexto: Hardware

8. Represente o código Java abaixo em um diagrama de classes UML, com atributos, métodos, visibilidades, o tipo de relacionamento e as multiplicidades. Explique por que o construtor de Processador não é public e como isso aparece no diagrama.

```
class Processador {
    private String modelo;
    private int nucleos;

    Processador(String modelo, int nucleos) {
        this.modelo = modelo;
        this.nucleos = nucleos;
    }
}

public class Computador {
    private String marca;
    private Processador processador;

    public Computador(String marca,
            String modeloCpu, int nucleos) {
        this.marca = marca;
        this.processador =
            new Processador(modeloCpu, nucleos);
    }
}
```

| | |
|---|---|
| | **Curso Superior em Análise e Desenvolvimento de Sistemas** |
| | Disciplina: Projeto de Software |
| | Profª. Denise do Rocio Maciel |
| | **LISTA DE EXERCÍCIOS: DIAGRAMA DE CLASSES UML** |

Faculdade Senac – 2026 – Projeto de Software – Contexto: Biblioteca

9. Represente o código Java abaixo em um diagrama de classes UML, com atributos, métodos, visibilidades, o tipo de relacionamento e as multiplicidades. Indique a diferença entre este caso e o exercício 7 quanto à multiplicidade do lado da Estante.

```
import java.util.*;

public class Livro {
    private String titulo;
    private Estante estante;

    public Livro(String titulo) {
        this.titulo = titulo;
    }

    void setEstante(Estante e) { estante = e; }
    public Estante getEstante() {
        return estante;
    }
}

public class Estante {
    private String setor;
    private List<Livro> livros =
        new ArrayList<>();

    public Estante(String setor) {
        this.setor = setor;
    }

    public void guardar(Livro l) {
        if (l.getEstante() != null)
            throw new IllegalStateException(
                "Livro em outra estante");
        livros.add(l);
        l.setEstante(this);
    }

    public void retirar(Livro l) {
        if (livros.remove(l))
            l.setEstante(null);
    }
}
```

Faculdade Senac – 2026 – Projeto de Software – Contexto: Vendas

10. Represente o código Java abaixo em um diagrama de classes UML, com atributos, métodos, visibilidades, o tipo de relacionamento e as multiplicidades. Que diferença em relação ao exercício 9 justifica o losango preenchido?

```
import java.util.*;

class ItemPedido {
    private String produto;
    private int quantidade;

    ItemPedido(String produto, int qtd) {
        this.produto = produto;
        this.quantidade = qtd;
    }
}

public class Pedido {
    private int numero;
    private List<ItemPedido> itens =
        new ArrayList<>();

    public Pedido(int numero) {
        this.numero = numero;
    }

    public void adicionarItem(String produto,
            int qtd) {
        itens.add(new ItemPedido(produto, qtd));
    }

    public void removerItem(int i) {
        itens.remove(i);
    }
}
```

| | |
|---|---|
| | **Curso Superior em Análise e Desenvolvimento de Sistemas** |
| | Disciplina: Projeto de Software |
| | Profª. Denise do Rocio Maciel |
| | **LISTA DE EXERCÍCIOS: DIAGRAMA DE CLASSES UML** |

Faculdade Senac – 2026 – Projeto de Software – Contexto: Escritório

11. Represente o código Java abaixo em um diagrama de classes UML, com atributos, métodos, visibilidades, o tipo de relacionamento e as multiplicidades. Por que não há multiplicidade nas pontas da seta?

```
public class Documento {
    private String conteudo;

    public Documento(String conteudo) {
        this.conteudo = conteudo;
    }

    public String getConteudo() {
        return conteudo;
    }
}

public class Impressora {
    private String modelo;

    public Impressora(String modelo) {
        this.modelo = modelo;
    }

    public void imprimir(Documento d) {
        System.out.println(d.getConteudo());
    }
}
```

Faculdade Senac – 2026 – Projeto de Software – Contexto: Relatórios gerenciais

12. Represente o código Java abaixo em um diagrama de classes UML, com atributos, métodos, visibilidades, o tipo de relacionamento e as multiplicidades. Como a multiplicidade do parâmetro deve ser escrita? Se a lista virasse atributo, qual seria o relacionamento?

```
import java.util.List;

public class Venda {
    private double valor;

    public Venda(double valor) {
        this.valor = valor;
    }

    public double getValor() { return valor; }
}

public class Relatorio {
    public double totalizar(List<Venda> vendas) {
        if (vendas.isEmpty())
            throw new IllegalArgumentException(
                "Informe ao menos uma venda");
        double total = 0;
        for (Venda v : vendas)
            total += v.getValor();
        return total;
    }
}
```

| | |
|---|---|
| | **Curso Superior em Análise e Desenvolvimento de Sistemas** |
| | Disciplina: Projeto de Software |
| | Profª. Denise do Rocio Maciel |
| | **LISTA DE EXERCÍCIOS: DIAGRAMA DE CLASSES UML** |



13. Represente o código Java abaixo em um diagrama de classes UML, com atributos, métodos, visibilidades, o tipo de relacionamento e as multiplicidades. Como representar a classe abstrata, o método abstrato e os atributos protected?

```
public abstract class Conta {
    protected String numero;
    protected double saldo;

    public Conta(String numero) {
        this.numero = numero;
    }

    public void depositar(double valor) {
        saldo += valor;
    }

    public abstract void sacar(double valor);
}

public class ContaCorrente extends Conta {
    private double limite;

    public ContaCorrente(String n, double lim) {
        super(n);
        this.limite = lim;
    }

    @Override
    public void sacar(double valor) {
        if (valor <= saldo + limite)
            saldo -= valor;
    }
}

public class ContaPoupanca extends Conta {
    private double taxa;

    public ContaPoupanca(String n, double t) {
        super(n);
        this.taxa = t;
    }

    @Override
    public void sacar(double valor) {
        if (valor <= saldo) saldo -= valor;
    }

    public void renderJuros() {
        saldo += saldo * taxa;
    }
}
```

| | |
|---|---|
| | **Curso Superior em Análise e Desenvolvimento de Sistemas** |
| | Disciplina: Projeto de Software |
| | Profª. Denise do Rocio Maciel |
| | **LISTA DE EXERCÍCIOS: DIAGRAMA DE CLASSES UML** |

Faculdade Senac – 2026 – Projeto de Software – Contexto: Pagamentos

14. Represente o código Java abaixo em um diagrama de classes UML, com atributos, métodos, visibilidades, o tipo de relacionamento e as multiplicidades. Qual a diferença visual entre esta seta e a do exercício 13?

```
public interface Pagavel {
    double calcularValor();
}

public class Boleto implements Pagavel {
    private double valor;
    private double multa;

    public Boleto(double valor, double multa) {
        this.valor = valor;
        this.multa = multa;
    }

    public double calcularValor() {
        return valor + multa;
    }
}

public class Cartao implements Pagavel {
    private double valor;
    private int parcelas;

    public Cartao(double valor, int parcelas) {
        this.valor = valor;
        this.parcelas = parcelas;
    }

    public double calcularValor() {
        return valor / parcelas;
    }
}
```

| | |
|---|---|
| | **Curso Superior em Análise e Desenvolvimento de Sistemas** |
| | Disciplina: Projeto de Software |
| | Profª. Denise do Rocio Maciel |
| | **LISTA DE EXERCÍCIOS: DIAGRAMA DE CLASSES UML** |

Faculdade Senac – 2026 – Projeto de Software – Contexto: Clínica (exercício integrador)

15. Represente o código Java abaixo em um diagrama de classes UML, com atributos, métodos, visibilidades, o tipo de relacionamento e as multiplicidades. Identifique TODOS os relacionamentos (herança, realização, composição, associação e dependência) e represente-os em um único diagrama.

```
import java.util.*;

public interface Notificavel {
    void notificar(String msg);
}

public abstract class Pessoa {
    protected String nome;
    public Pessoa(String nome) {
        this.nome = nome;
    }
}

class Prontuario {
    private List<String> anotacoes =
        new ArrayList<>();
    Prontuario() { }
}

public class Paciente extends Pessoa
        implements Notificavel {
    private Prontuario prontuario;

    public Paciente(String nome) {
        super(nome);
        prontuario = new Prontuario();
    }

    public void notificar(String msg) {
        System.out.println(nome + ": " + msg);
    }
}

public class Receita {
    private String texto;
    public Receita(String texto) {
        this.texto = texto;
    }
    public String getTexto() { return texto; }
}

public class Medico extends Pessoa {
    private String crm;
    private List<Paciente> pacientes =
        new ArrayList<>();

    public Medico(String nome, String crm) {
        super(nome);
        this.crm = crm;
    }

    public void atender(Paciente p) {
        if (!pacientes.contains(p))
            pacientes.add(p);
    }

    public void emitir(Receita r, Paciente p) {
        p.notificar(r.getTexto());
    }
}
```

| | |
|---|---|
| | **Curso Superior em Análise e Desenvolvimento de Sistemas** |
| | Disciplina: Projeto de Software |
| | Profª. Denise do Rocio Maciel |
| | **LISTA DE EXERCÍCIOS: DIAGRAMA DE CLASSES UML** |

